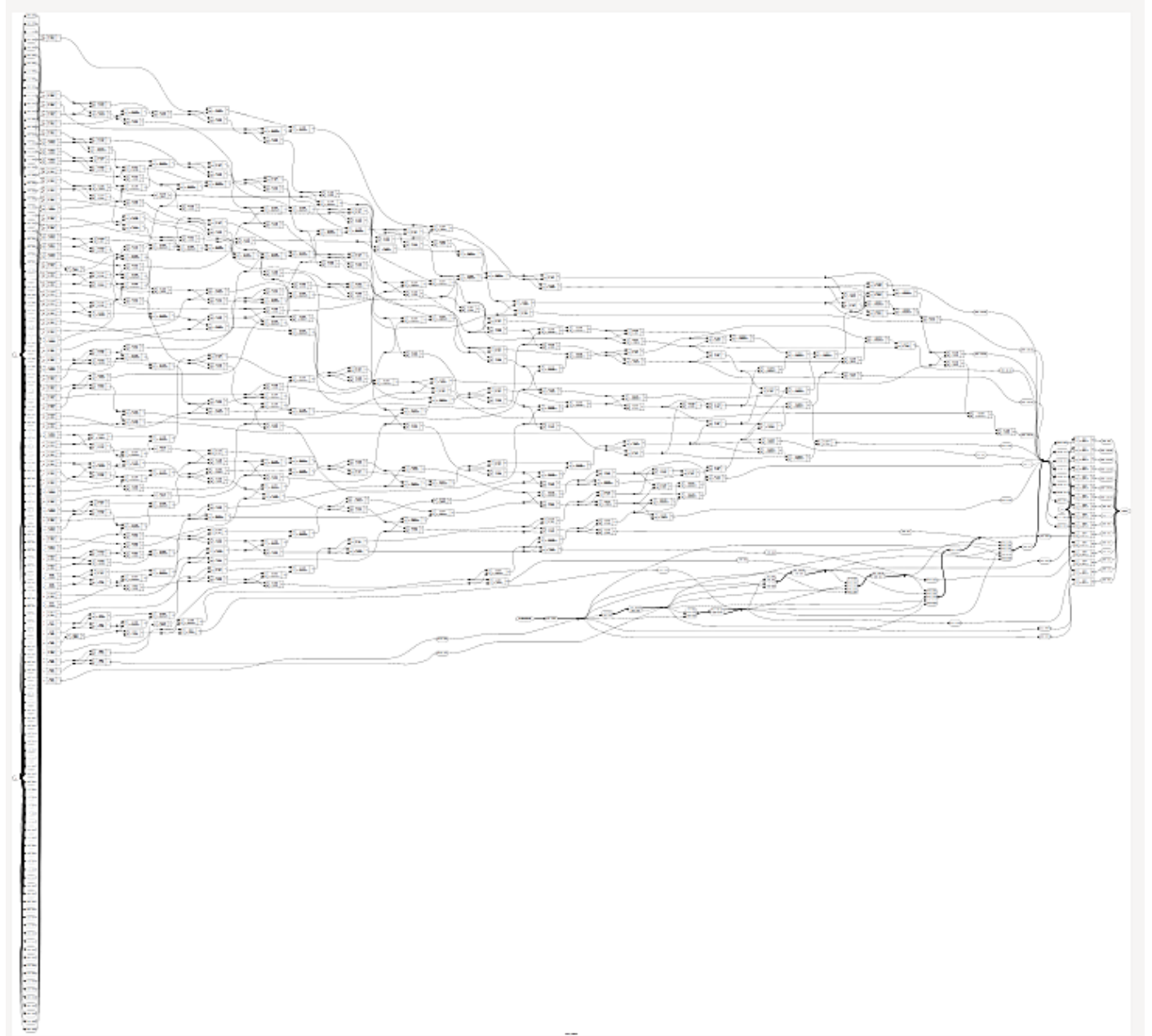


Day 11

Multiplier without pipeline:

Verilog Code:

```
module mult_nopipe(  
    input    clk,  
    input  [7:0] a, b,  
    output reg [15:0] result  
);  
    always @(posedge clk)  
        result <= a * b;  
endmodule
```



2.26. Printing statistics.

=== mult_nopipe ===

Number of wires:	349
Number of wire bits:	483
Number of public wires:	4

Number of public wire bits:	33
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	370
\$_ANDNOT_	97
\$_AND_	55
\$_DFF_P_	16
\$_NAND_	25
\$_NOR_	12
\$_NOT_	7
\$_ORNOT_	10
\$_OR_	37
\$_XNOR_	25
\$_XOR_	86

Multiplier with pipeline:

Verilog code:

```

module mult_pipe2(
    input clk,
    input [7:0] a, b,
    output reg [15:0] result
);

    // Pipeline Registers
    reg [7:0] a_reg, b_reg;
    reg [15:0] partial [0:7];
    integer i;

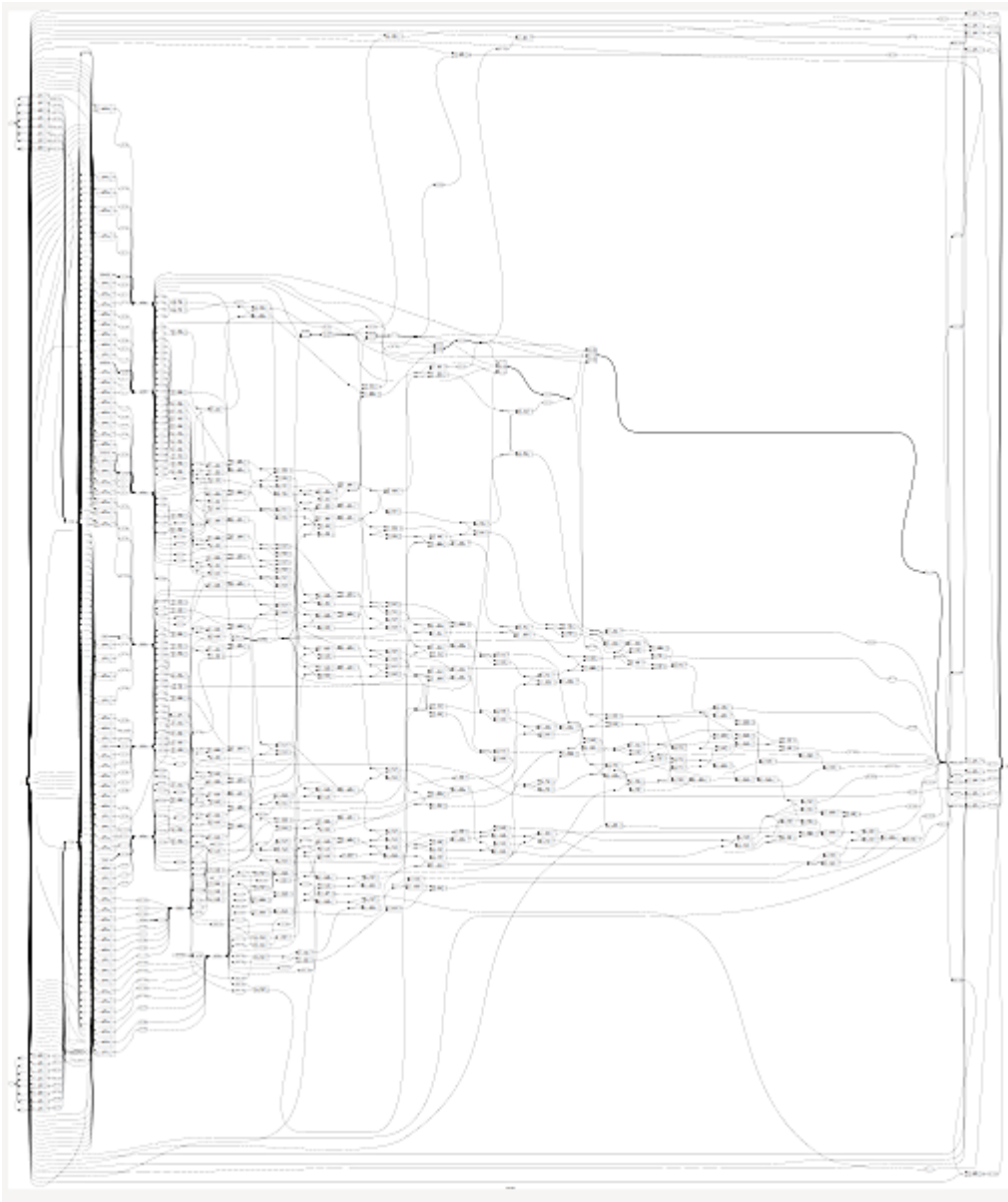
    // STAGE 1: Synchronous Data Capture & Partial Product Generation
    always @(posedge clk) begin
        // Register inputs to ensure they stay stable for the calculation
        a_reg <= a;
        b_reg <= b;

        // Generate partial products using the REGISTERED inputs
        // This ensures Stage 1 data is isolated from the raw inputs
        for (i = 0; i < 8; i = i + 1) begin
            if (b_reg[i])
                partial[i] <= (a_reg << i);
            else
                partial[i] <= 16'b0;
        end
    end

    // STAGE 2: Summation of Registered Partial Products
    always @(posedge clk) begin
        // Summing the partial products calculated in the previous cycle
        result <= partial[0] + partial[1] + partial[2] + partial[3] +
            partial[4] + partial[5] + partial[6] + partial[7];
    end

endmodule

```



6. Printing statistics.

=== mult_pipe2 ===

Number of wires:	301
Number of wire bits:	539
Number of public wires:	14
Number of public wire bits:	163
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	392
\$_ANDNOT_	103
\$_AND_	7
\$_DFF_P_	32
\$_NAND_	17
\$_NOR_	8
\$_NOT_	5
\$_ORNOT_	8
\$_OR_	31
\$_SDFF_PN0_	64
\$_XNOR_	37
\$_XOR_	80

ALU with pipelining:

```
module alu_pipe(
    input    clk, rst,
    input  [7:0] a, b,
    input  [3:0] opcode,
    output reg [7:0] result, // registered output
    output reg    zero,
    output reg    carry,
    output reg    negative,
    output reg    overflow
);
    // Stage 1 — combinational ALU (same as Day 10)
    reg [7:0] s1_result;
    reg    s1_carry;
    reg [7:0] s1_a, s1_b;
    reg [3:0] s1_opcode;

    // Your ALU logic here (copy from Day 10)
    reg carry_out;
    always @(*) begin
        carry_out = 0;
        case(opcode)
            4'b0000: {carry_out, s1_result} = a + b;
            4'b0001: {carry_out, s1_result} = a - b;
            4'b0010: s1_result = a & b;
            4'b0011: s1_result = a | b;
            4'b0100: s1_result = a ^ b;
            4'b0101: s1_result = ~a;
            4'b0110: {carry_out, s1_result} = {a, 1'b0};
            4'b0111: {s1_result, carry_out} = {1'b0, a};
            default: s1_result = 0;
        endcase
        s1_carry = carry_out;
    end

    // Pipeline register — stage 1 to stage 2
    always @(posedge clk) begin
        if(rst) begin
            s1_a    <= 0;
            s1_b    <= 0;
            s1_opcode <= 0;
        end
        else begin
            s1_a    <= a;
            s1_b    <= b;
            s1_opcode <= opcode;
        end
    end

    // Stage 2 — flag computation on registered result
    always @(posedge clk) begin
        if(rst) begin
            result <= 0;
            zero   <= 0;
            carry  <= 0;
            negative <= 0;
        end
    end
end
```

```

        overflow <= 0;
    end
    else begin
        result <= s1_result;
        carry <= s1_carry;
        zero <= (s1_result == 0);
        negative <= s1_result[7];
        overflow <= (s1_opcode == 4'b0000) ?
            (~s1_a[7] & ~s1_b[7] & s1_result[7]) |
            ( s1_a[7] & s1_b[7] & ~s1_result[7]) :
            (s1_opcode == 4'b0001) ?
            (~s1_a[7] & s1_b[7] & s1_result[7]) |
            ( s1_a[7] & ~s1_b[7] & ~s1_result[7]) : 0;
    end
end
endmodule

```

Testbench:

```

`timescale 1ns/1ps
module tb;
    reg    clk, rst;
    reg [7:0] a, b;
    reg [3:0] opcode;
    wire [7:0] result;
    wire    zero, carry, negative, overflow;

    alu_pipe uut(.clk(clk), .rst(rst),
        .a(a), .b(b), .opcode(opcode),
        .result(result), .zero(zero),
        .carry(carry), .negative(negative),
        .overflow(overflow));

    initial begin clk = 0; forever #5 clk = ~clk; end

    initial begin
        $dumpfile("alu_pipe.vcd");
        $dumpvars(0, tb);
    end

    initial begin
        rst = 1; a = 0; b = 0; opcode = 0;
        #20; rst = 0;

        // Cycle 1 — feed ADD
        @(negedge clk); a = 8'h0F; b = 8'h01; opcode = 4'b0000;
        $display("Cycle 1 input: ADD 0F + 01");

        // Cycle 2 — feed SUB, ADD result not ready yet
        @(negedge clk); a = 8'hAA; b = 8'hAA; opcode = 4'b0001;
        $display("Cycle 2 input: SUB AA - AA | result=%h (pipeline filling)", result);

        // Cycle 3 — ADD result now appears
        @(negedge clk); a = 8'hF0; b = 8'h0F; opcode = 4'b0011;
        $display("Cycle 3 input: OR | result=%h (should be 10 = ADD result)", result);

        // Cycle 4 — SUB result appears
        @(negedge clk); a = 0; b = 0; opcode = 0;
    end
endmodule

```

```
$display("Cycle 4    | result=%h (should be 00 = SUB result)", result);
```

```
// Cycle 5 — OR result appears
```

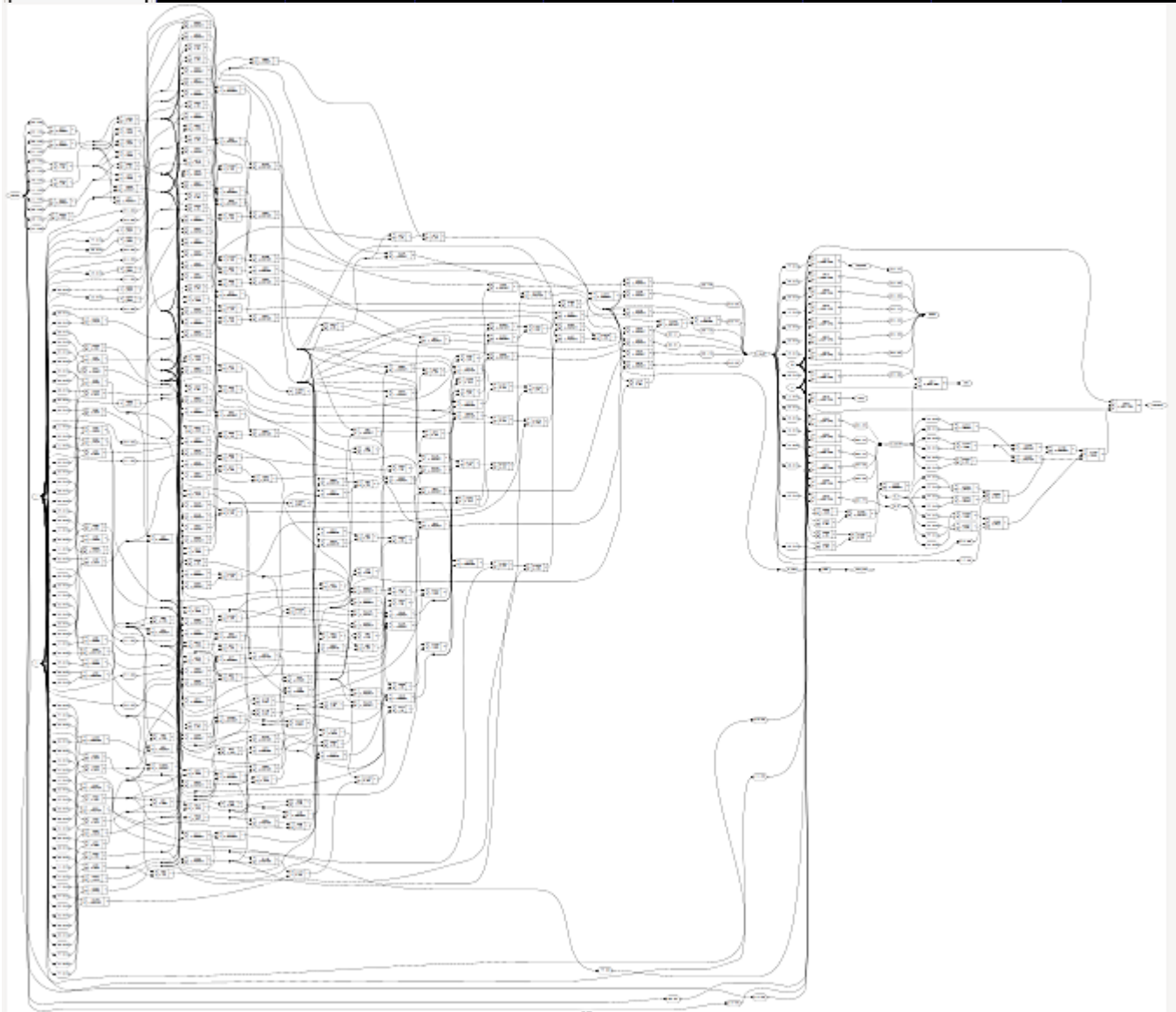
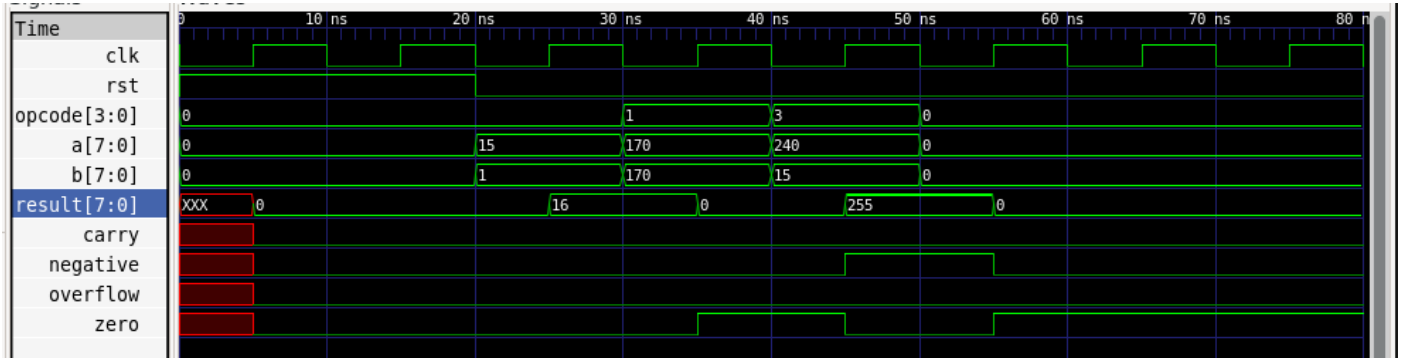
```
@(negedge clk);
```

```
$display("Cycle 5    | result=%h (should be FF = OR result)", result);
```

```
#20; $finish;
```

```
end
```

```
endmodule
```

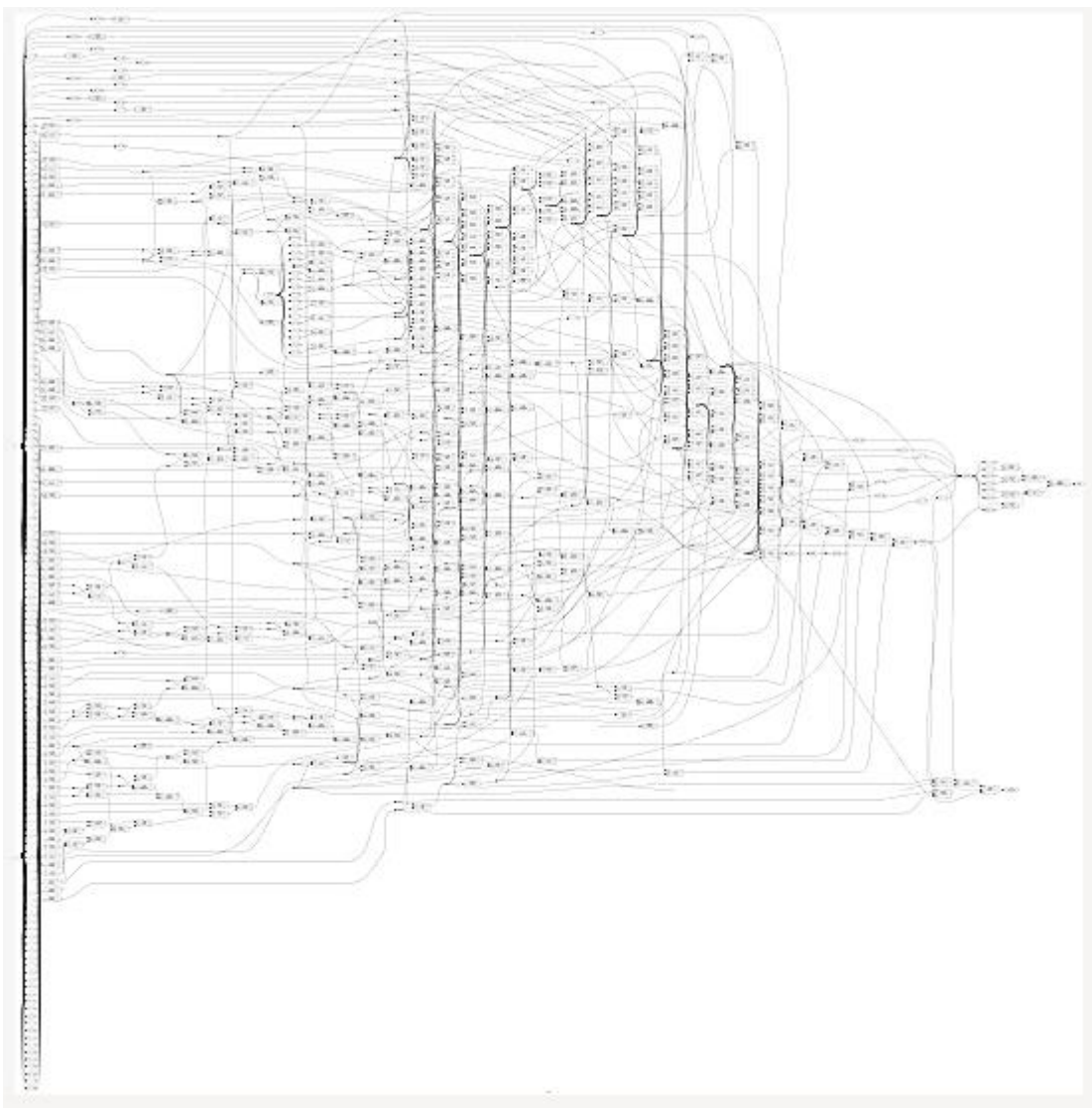


3. Printing statistics.

```
=== alu_pipe ===
```

Number of wires:	296
Number of wire bits:	344
Number of public wires:	16
Number of public wire bits:	64
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	306
\$_ANDNOT_	127
\$_AND_	13
\$_MUX_	3
\$_NAND_	4
\$_NOR_	26
\$_NOT_	10
\$_ORNOT_	13
\$_OR_	71
\$_SDFF_PP0_	17
\$_XNOR_	7
\$_XOR_	15

ALU without pipelining



3. Printing statistics.

=== alu ===

Number of wires:	468
Number of wire bits:	492
Number of public wires:	9
Number of public wire bits:	33
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	470
\$_ANDNOT_	112
\$_AND_	41
\$_MUX_	102
\$_NAND_	19
\$_NOR_	28
\$_NOT_	27
\$_ORNOT_	23
\$_OR_	46
\$_XNOR_	9
\$_XOR_	63

Two things stand out:

1. Non-pipelined ALU has 102 \$_MUX_ cells vs pipelined's 3.

The non-pipelined version is purely combinational — every opcode path feeds into a large mux tree driving the output. The pipelined version registers the intermediate result, so stage 2 just picks up the already-computed value — far fewer muxes needed.

2. Pipelined ALU has fewer total cells (306 vs 470) despite having FFs.

This seems counterintuitive — adding registers made the design *smaller*? Yes, because the pipeline stage boundary lets Yosys optimise each stage independently. The combinational cloud in the non-pipelined version is one huge unoptimised blob. Split into two stages, the optimiser works better on each half.

The three core lessons of Day 11:

1. Pipelining trades latency for throughput and clock speed

- Latency goes up by 1 cycle per stage
- Throughput stays at 1 result/cycle once full
- Clock can run faster because each stage has less logic depth

2. Pipeline registers cost FFs but can reduce total logic

- Stage boundaries allow better optimisation per stage
- The ALU actually got smaller with pipelining — 470 → 306 cells

3. \$_SDFF_PNO_ is the pipeline register cell

- S = synchronous reset, P = positive clock, P = positive reset polarity, 0 = resets to 0
- These are the if(rst) ... <= 0 pipeline flush registers
- 17 of them in the pipelined ALU = 8 bits result + 8 bits a/b forwarded + 1 carry